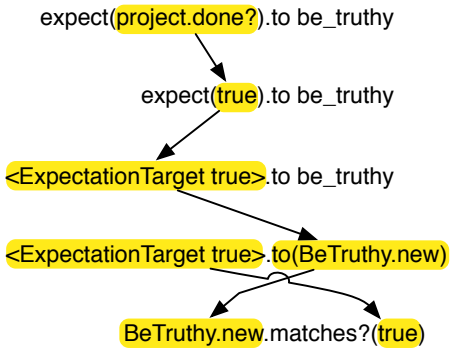
There's nothing special about an RSpec matcher; at base it's just an object that responds to a matches? method. There are several predefined matchers and you can write your own.

In this case, be_truthy is a method defined by RSpec to return the BeTruthy matcher. You could get the same behavior with

```
expect(project.done?).to(RSpec::BuiltIn::BeTruthy.new)
```

but you probably would agree that the idiomatic version is easier to read.

The ExpectationTarget is now holding on to two objects: the object being matched (in this case, project.done?) and the matcher (be_truthy). When the spec is executed, RSpec calls the matches? method on the matcher, with the object being matched as an argument. If the expectation uses to, then the expectation passes if matches? is true. If the expectation uses not_to, then it checks for a does_not_match? method in the matcher. If there is no such method it falls back to passing if matches? is false. This is shown in the following diagram.



From an RSpec perspective, you're creating an object and asserting an initial condition. What are you doing from a TDD perspective, and why is this useful?

Small as it might seem, you've performed a little bit of design. You're starting to define the way parts of your system communicate with each other, and the tests ensure the visibility of important information in your design.

This small test makes three claims about your program:

- There is a class called Project.
- You can query instances of that class as to whether they are done.
- A brand-new instance of Project qualifies as done.

This last assertion isn't inevitable—you could say that you aren't done unless there is at least one completed task, but that's a choice you're making in the application's business logic.